MODUL 3

KONEKTIVITAS MQTT DENGAN MICROPYTHON, RASPBERRY PI PICO W, DAN ECLIPSE MOSQUITTO

TUJUAN

1) Mengetahui protokol MQTT.
2) Memahami proses komunikasi protokol MQTT dengan broker lokal.
3) Memahami implementasi IoT menggunakan Python pada mikrokontroler Raspberry Pi Pico W.

REQUIREMENT

1) Thonny sebagai IDE MicroPython,
2) Raspberry Pi Pico W,
3) Server Lokal (Raspberry Pi 3B).
4) Pemahaman dasar pemrograman Python.

3.1 MQTT

MQTT adalah protokol transport pesan client-server berbasis publish-subscribe yang dirancang ringan untuk lingkungan dengan keterbatasan bandwidth, daya, atau sumber daya perangkat. Client berkomunikasi melalui broker menggunakan topic. MQTT mendukung beberapa tingkat Quality of Service (QoS), sesi, retained message, dan Last Will sesuai versi protokol serta fitur yang digunakan.

3.2 Kelebihan MQTT

MQTT banyak digunakan pada sistem IoT karena beberapa karakteristik berikut:

a. **Ringan dan efisien**

Penggunaan MQTT pada sistem IoT dapat diimplementasikan pada mikrokontroler kecil dikarenakan kecilnya data yang digunakan. Dengan kecilnya data tersebut, maka bandwidth dalam jaringan dapat dioptimalkan.

b. **Skalabel**

Arsitektur broker dan pemisahan antara publisher serta subscriber memungkinkan sistem dikembangkan untuk melayani banyak client. Skalabilitas aktual tetap bergantung pada kapasitas broker, desain topic, ukuran pesan, QoS, jaringan, dan pola koneksi.

c. **Andal**

MQTT menyediakan QoS 0, QoS 1, dan QoS 2 dengan jaminan pengiriman yang berbeda. Keandalan tidak sama dengan latensi: QoS yang lebih tinggi dapat menambah pertukaran paket dan overhead. Pemilihan QoS harus disesuaikan dengan risiko kehilangan atau duplikasi pesan.

d. **Didukung secara luas**

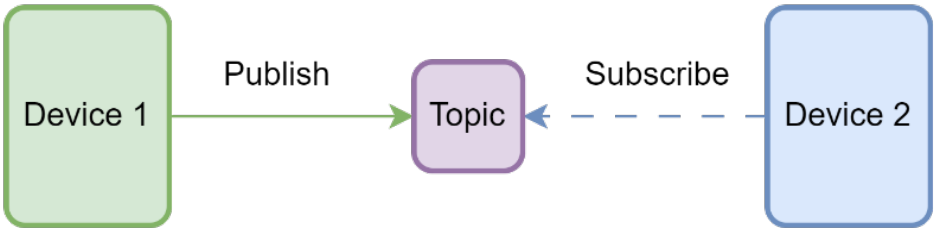
MQTT didukung oleh banyak bahasa pemrograman dan platform, termasuk Python yang memiliki library khusus untuk implementasi protokol MQTT pada perangkat IoT. Oleh karena itu, pengembang dapat mengembangkan perangkat IoT dengan optimal.

3.3 Alur Komunikasi MQTT

Seperti yang telah disebutkan sebelumnya, komunikasi MQTT menerapkan sistem **Publish-Subscribe** untuk melakukan proses transfer data. Pertukaran data antar perangkat yang dilakukan oleh MQTT merupakan **pesan/messages** yang berisi informasi seperti pembacaan sensor atau perintah.

3.3.1 Publish-Subscribe

Istilah publish dan subscribe pada MQTT digunakan saat client bertukar pesan melalui broker. Publisher mengirimkan pesan ke topic tertentu, sedangkan subscriber berlangganan menggunakan topic filter agar broker meneruskan pesan yang sesuai.



Gambar 3.1: Visualisasi Publish - Subscribe

Seperti yang ditunjukkan pada Gambar 3.1, Device 1 memublikasikan pesan ke suatu topic dan Device 2 berlangganan pada topic filter yang sesuai. Broker menerima pesan dari Device 1, kemudian meneruskannya kepada Device 2.

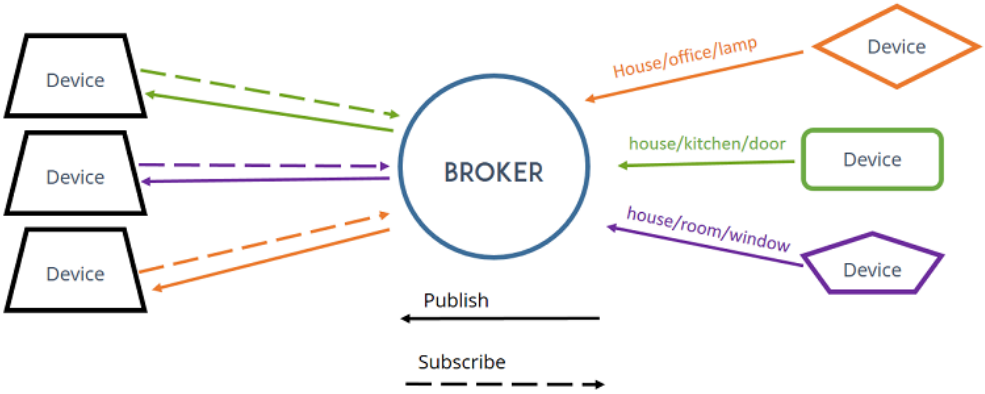
3.3.2 Topics

Topic adalah string hierarkis yang digunakan broker untuk mengarahkan pesan. Client mempublikasikan pesan ke suatu topic, sedangkan subscriber menggunakan topic filter untuk menerima pesan yang sesuai. Topic bersifat case-sensitive; wildcard + dan # dapat digunakan pada subscription sesuai aturan MQTT, tetapi tidak digunakan sebagai nama topic ketika melakukan publish.

Topic MQTT berupa string dan bersifat **case-sensitive**, sehingga perbedaan huruf kapital menghasilkan nama topic yang berbeda.

3.3.3 Broker

Broker menerima pesan dari publisher, mencocokkannya dengan subscription, lalu meneruskannya kepada client yang topic filternya sesuai. Broker juga dapat mengelola sesi, QoS, retained message, autentikasi, dan otorisasi sesuai konfigurasi.



Gambar 3.2: Visualisasi Broker

Tersedia berbagai layanan broker MQTT yang tersedia di internet, misalnya Adafruit IO dan HiveMQ, dengan pilihan paket serta batas penggunaan yang berbeda. Broker juga dapat dijalankan sendiri pada mikrokomputer seperti Raspberry Pi menggunakan software Mosquitto.

3.4 Eclipse Mosquitto

Eclipse Mosquitto adalah message broker MQTT sumber terbuka yang ringan dan dapat dijalankan pada berbagai sistem, termasuk Raspberry Pi. Dengan Mosquitto, broker lokal dapat disiapkan tanpa bergantung pada layanan broker eksternal seperti Adafruit IO dan HiveMQ.



Gambar 3.3: Logo Mosquitto MQTT

3.5 MicroPython

MicroPython adalah implementasi Python yang dioptimalkan untuk berjalan di mikrokontroler dan perangkat berdaya rendah. Diciptakan oleh Damien George, MicroPython memungkinkan pengembang untuk menggunakan bahasa pemrograman Python pada perangkat keras yang memiliki sumber daya terbatas, seperti mikrokontroler, mikroprosesor, dan perangkat Internet of Things (IoT).

Berikut adalah beberapa fitur dan karakteristik utama dari MicroPython:

a. **Ringan dan Optimal**

MicroPython dirancang untuk berjalan pada perangkat keras yang memiliki sumber daya terbatas. Interpreter dan inti bahasa Python diperkecil untuk memastikan kinerja yang optimal pada mikrokontroler dengan kapasitas memori yang rendah.

b. **Dukungan untuk Mikrokontroler Populer**

MicroPython mendukung sejumlah mikrokontroler populer seperti ESP8266, ESP32, STM32, dan sebagainya. Ini memungkinkan pengembang untuk menggunakan bahasa Python pada berbagai perangkat keras yang umum digunakan dalam proyek mikrokontroler.

c. **Interaktif**

Salah satu fitur unik dari MicroPython adalah mode interaktifnya. Pengguna dapat berinteraksi langsung dengan perangkat keras menggunakan REPL (read-eval-print loop) yang mirip dengan Python pada komputer desktop. Hal ini memudahkan pengembangan dan debugging.

d. **Dukungan Modul Standar Python**

Meskipun dibuat untuk mikrokontroler, MicroPython mencakup sejumlah modul standar Python yang dapat ditemui di implementasi Python lainnya. Meskipun modul tersebut dapat berbeda dalam ukuran atau implementasi khusus, banyak modul standar dapat digunakan di MicroPython tanpa modifikasi.

e. **Kemampuan I/O**

MicroPython menyediakan antarmuka untuk mengakses pin I/O dan perangkat keras lainnya pada mikrokontroler, memungkinkan pengembang untuk mengendalikan sensor, motor, dan perangkat keras lainnya.

f. **Dukungan Wi-Fi dan Jaringan**

Beberapa implementasi MicroPython, terutama pada mikrokontroler seperti ESP8266 dan ESP32, menyediakan dukungan untuk konektivitas Wi-Fi dan jaringan, memungkinkan perangkat untuk terhubung dengan internet.

g. **Komunitas dan Ekosistem yang Berkembang**

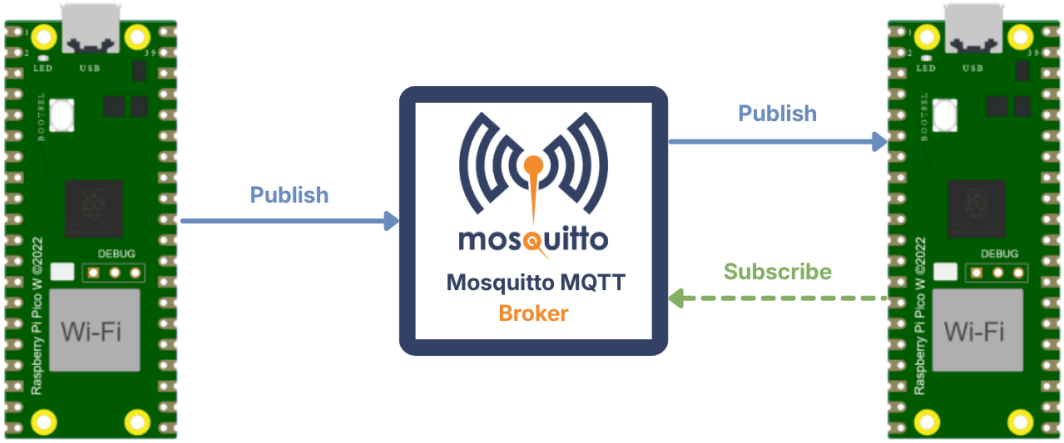
MicroPython memiliki komunitas pengembang yang aktif, dan banyak sumber daya dan tutorial yang tersedia secara online. Ini membuatnya lebih mudah bagi pengembang untuk memulai dengan MicroPython dan memecahkan masalah yang mereka hadapi.

MicroPython memberikan kemudahan pengembangan dan kekuatan bahasa Python pada mikrokontroler, membuatnya menjadi pilihan yang menarik untuk proyek-proyek IoT dan pengembangan perangkat keras berdaya rendah.

3.6 Raspberry Pi Pico W

Raspberry Pi Pico W adalah papan mikrokontroler berbasis RP2040 dengan prosesor dual-core Arm Cortex-M0+, SRAM 264 kB, dan flash eksternal 2 MB pada papan standar. Pico W menambahkan konektivitas Wi-Fi 2,4 GHz IEEE 802.11 b/g/n dan dukungan Bluetooth 5.2 melalui chip nirkabel yang berkomunikasi dengan RP2040. Papan ini dapat diprogram menggunakan MicroPython, C, atau C++; MicroPython bukan satu-satunya bahasa pemrograman yang didukung. Diagram pinout dapat dilihat pada Lampiran 3.

3.7 Implementasi Protokol MQTT



Gambar 3.4: Implementasi MQTT dengan Raspberry Pi Pico W

Berikut alur pengiriman antara Raspberry Pi Pico W dan Mosquitto MQTT:

a. Device 1 memublikasikan pesan ke topic 'A' pada broker MQTT.
b. Device 2 berlangganan pada topic 'A'.
c. Broker MQTT meneruskan pesan dari Device 1 kepada Device 2 sesuai langganannya.

3.7.1 Setup Broker Lokal dengan Raspberry Pi

Untuk menggunakan MQTT, diperlukan broker sebagai perantara antara client yang memublikasikan pesan dan client yang berlangganan. Di sini, komputer papan tunggal Raspberry Pi digunakan untuk menjalankan broker MQTT lokal. Perlu diketahui bahwa bagian ini bersifat opsional bila Anda menggunakan broker eksternal seperti HiveMQ atau AdafruitIO.

Instal paket `mosquitto` dalam Raspberry Pi sebagai berikut di dalam terminal:

Instalasi Mosquitto MQTT Broker

```
sudo apt update
sudo apt install mosquitto mosquitto-clients
```

Saat menjalankan perintah dengan `sudo`, sistem mungkin meminta kata sandi akun Raspberry Pi. Kita akan menggunakan paket `mosquitto` dan `mosquitto-clients` untuk melakukan setup pada broker MQTT ini.

Setelah itu, jalankan kode berikut di dalam terminal untuk mengaktifkan mosquitto ketika Raspberry Pi dinyalakan:

Mengaktifkan autostart pada Mosquitto MQTT

```
sudo systemctl enable mosquitto.service
```

Untuk melihat alamat IP Raspberry Pi, jalankan perintah berikut di terminal:

Melihat IP Address dari Raspberry Pi

```
hostname -I
# Atau Anda bisa menggunakan perintah berikut
ip addr
```

Program di atas akan menampilkan alamat IPv4 dan IPv6 Raspberry Pi. Kita hanya memerlukan alamat IPv4 untuk broker kita, sebagai contoh `192.168.1.100`.

3.7.1.1 Tanpa Autentikasi

Agar broker dapat diakses dari perangkat lain tanpa autentikasi, kita perlu mengedit konfigurasi pada mosquitto yang akan digunakan. Edit file `mosquitto.conf` yang berada di `/etc/mosquitto/mosquitto.conf` dengan teks editor yang Anda gunakan, contohnya teks editor `nano` di dalam terminal:

Membuka teks editor di dalam terminal

```
sudo nano /etc/mosquitto/mosquitto.conf
```

Ketika pada mode pengeditan, tambahkan beberapa opsi berikut pada bagian bawah baris konfigurasi `mosquitto.conf`:

Konten file konfigurasi dari mosquitto.conf

```
# Konfigurasi Default
pid_file /run/mosquitto/mosquitto.pid

persistence true
persistence_location /var/lib/mosquitto

log_dest file /var/log/mosquitto/mosquitto.log

include_dir /etc/mosquitto/conf.d

# Konten baru dari file
listener 1883
allow_anonymous true
```

Pada konfigurasi di atas, konfigurasi `listener 1883` akan mengatur port yang dipakai oleh MQTT, sedangkan `allow_anonymous true` memperbolehkan siapa pun untuk menggunakan broker tersebut tanpa autentikasi. Konfigurasi ini hanya boleh digunakan pada jaringan praktikum yang terisolasi dan tidak boleh diekspos ke internet. Setelah menyimpan konfigurasi, lakukan restart pada service Mosquitto.

Memulai ulang service pada program Mosquitto MQTT

```
sudo systemctl restart mosquitto.service
```

3.7.2 Dengan Autentikasi

Bila Anda ingin menambahkan autentikasi pada broker, diperlukan beberapa konfigurasi tambahan. Buatlah nama pengguna dan kata sandi yang akan digunakan pada broker MQTT.

Menambahkan User dan Password pada program Mosquitto MQTT

```
sudo mosquitto_passwd -c /etc/mosquitto/passwd YOUR_USERNAME
# Setelah menjalankan perintah di atas, masukkan kata sandi saat diminta.

# Ubah permission filenya agar program Mosquitto MQTT dapat membaca filenya
sudo chown mosquitto:mosquitto /etc/mosquitto/passwd
sudo chmod 600 /etc/mosquitto/passwd
```

Saat memasukkan kata sandi, terminal biasanya tidak menampilkan karakter yang diketik. Pastikan kata sandi dicatat dan disimpan dengan aman. Setelah itu, ubah konfigurasi default dari `mosquitto.conf` menjadi sebagai berikut:

Konten file konfigurasi dari mosquitto.conf

```
# Konten baru dari file
# Konfigurasi autentikasi berlaku untuk listener yang digunakan

# Konfigurasi Default
```

```
pid_file /run/mosquitto/mosquitto.pid

persistence true
persistence_location /var/lib/mosquitto

log_dest file /var/log/mosquitto/mosquitto.log

include_dir /etc/mosquitto/conf.d

# Konten baru dari file
allow_anonymous false
listener 1883
password_file /etc/mosquitto/passwd
```

Berbeda dengan sebelumnya, `allow_anonymous` diatur menjadi `false` agar client tanpa kredensial yang valid tidak dapat menggunakan broker. Autentikasi nama pengguna dan kata sandi tidak mengenkripsi lalu lintas; gunakan TLS apabila broker diakses melalui jaringan yang tidak tepercaya. Berikutnya kita konfigurasi file `passwd` agar program Mosquitto MQTT dapat membaca dengan perintah berikut:

Mengubah permission file dari passwd

```
sudo chown mosquitto:mosquitto /etc/mosquitto/passwd
sudo chmod 600 /etc/mosquitto/passwd
```

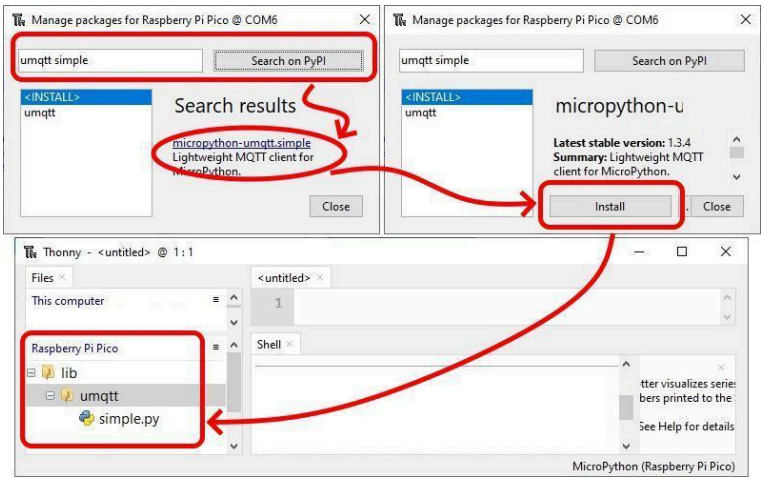
Terakhir, lakukan restart pada service dari program mosquitto.

Memulai ulang service pada program Mosquitto MQTT

```
sudo systemctl restart mosquitto.service
```

3.7.3 Setup Library MQTT pada Raspberry Pi Pico W

Library yang digunakan untuk menjalankan MQTT pada Pico W adalah `micropython-umqtt.simple`. Berikut adalah cara untuk menambahkan library pada Thonny untuk MicroPython:



Gambar 3.5: Tools > Manage Packages

3.7.4 Source Code Raspberry Pi Pico W

3.7.4.1 Publish

a. **Kebutuhan library dan variabel**

Library yang dibutuhkan untuk menggunakan protokol MQTT pada MicroPython adalah library `umqtt.simple`.

MicroPython Library Import

```
import network                      # Untuk mengkoneksikan ke internet
import time                         # Untuk fungsi sleep()
from umqtt.simple import MQTTClient # Library MQTT
import utime                        # Formatting Waktu
```

Setelah import library, maka langkah selanjutnya yang akan dilakukan adalah mendeklarasikan variabel yang akan digunakan. Variabel yang digunakan antara lain:

- Variabel untuk jaringan Wi-Fi
- Variabel untuk MQTT client
- Variabel yang akan dipublikasikan ke Broker

Source code untuk inisialisasi variabel

```
# Isi nama Wi-Fi dan password yang terkoneksi bersama broker lokal di sini
wifi_ssid     = "<YOUR SSID WIFI>"
wifi_password = "<YOUR WIFI PASSWORD>"

# Isi sesuai dengan broker yang akan dipakai (Mosquitto, HiveMQ, AdafruitIO)
# dan lain-lain
mqtt_host     = "<MQTT HOST ADDRESS>"

# (Opsional) Bila menggunakan autentikasi
mqtt_username = "<USERNAME FROM MOSQUITTO MQTT>"
mqtt_password = "<PASSWORD FROM MOSQUITTO MQTT>"

# Isi sesuai dengan topik yang ingin di-publish
mqtt_publish_topic = "<YOUR mqtt_topic/feed_director>"

# Isi ID client random (harus unik tiap client)
mqtt_client_id = "uus9asndajsndiqiwd10wid01n1kwd"

# Angka yang akan dipublish
counter = 0
```

b. Menghubungkan Pico W ke Wi-Fi dan MQTT server

Setelah variabel dideklarasikan, maka langkah selanjutnya adalah dengan menghubungkan Pico W ke Wi-Fi dan MQTT server. Fungsi yang digunakan untuk terhubung ke Wi-Fi adalah:

```
wlan.connect("<YOUR WIFI SSID>", "<YOUR WIFI PASSWORD>")
```

dan fungsi untuk terhubung ke MQTT client adalah:

```
mqtt_client.connect()
```

Source Code untuk terhubung ke Wi-Fi dan MQTT server

```
# Koneksi ke WiFi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(wifi_ssid, wifi_password)

while wlan.isconnected() == False:
  print('Waiting for connection...')
  time.sleep(1)

print("Connected to WiFi")

# Initialize our MQTTClient and connect to the MQTT server
mqtt_client = MQTTClient(
  client_id=mqtt_client_id,
  server=mqtt_host,
```

```
  # Bila menggunakan autentikasi
  user=mqtt_username,
  password=mqtt_password
)

mqtt_client.connect()
```

c. **Kode block try**

Pada kode di bawah ini, maka Pico W akan melakukan publish informasi dari variabel `count` menggunakan fungsi:

```
mqtt_client.publish("<Topic yang dituju>", str(<Nilai yang akan dipublikasikan>))
```

Source Code untuk block try (seperti fungsi loop pada Arduino)

```
try:
  while True:
    # Generate some dummy data that changes every loop
    counter += 3

    # Get the current timestamp
    current_time = utime.localtime()
    timestamp = "{:04}-{:02}-{:02} {:02}:{:02}:{:02}".format(
      current_time[0], current_time[1], current_time[2],
      current_time[3], current_time[4], current_time[5]
    )

    # Publish the data to the topic!
    print(f'{timestamp} - Publish {counter}')
    mqtt_client.publish(mqtt_publish_topic, str(counter))

    # Delay a bit to avoid hitting the rate limit
    time.sleep(3)

# Jika ada masalah pada saat pengiriman
except Exception as e:
  print(f'MQTT error: {e}')

# Ketika program python dimatikan
finally:
  mqtt_client.disconnect()
```

Pengiriman/publish informasi ke server MQTT tersebut akan berulang setiap 3 detik menggunakan fungsi `time.sleep(3)`.

3.7.4.2 Subscribe

Sama seperti sebelumnya, library yang dibutuhkan untuk melakukan subscribe pada mqtt di MicroPython adalah library `umqtt.simple`

MicroPython Library Import

```
import network
import time
from umqtt.simple import MQTTClient
import utime
```

Source code untuk inisialisasi variabel

```
# Isi nama Wi-Fi dan password yang terkoneksi bersama broker lokal di sini
wifi_ssid     = "<YOUR SSID WIFI>"
wifi_password = "<YOUR WIFI PASSWORD>"

# Isi sesuai dengan broker yang akan dipakai (Mosquitto, HiveMQ, AdafruitIO)
```

```
# dan lain-lain
mqtt_host     = "<MQTT HOST ADDRESS>"

# (Opsional) Bila menggunakan autentikasi
mqtt_username = "<USERNAME FROM MOSQUITTO MQTT>"
mqtt_password = "<PASSWORD FROM MOSQUITTO MQTT>"

# Isi sesuai dengan topik yang ingin di-subscribe
mqtt_receive_topic = "<YOUR mqtt_topic/feed_director>"

# Isi ID client random (harus unik tiap client)
mqtt_client_id = "iij0dfq8f012nif2jnfijn"

```

Source Code untuk terhubung ke Wi-Fi dan MQTT server

```
# Koneksi ke WiFi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(wifi_ssid, wifi_password)

while wlan.isconnected() == False:
  print('Waiting for connection...')
  time.sleep(1)

print("Connected to WiFi")

# Initialize our MQTTClient and connect to the MQTT server
mqtt_client = MQTTClient(
  client_id=mqtt_client_id,
  server=mqtt_host,
  # Bila menggunakan autentikasi
  user=mqtt_username,
  password=mqtt_password
)
```

Perbedaan kode sumber subscribe dan publish terletak pada fungsi callback yang akan dipanggil setiap ada informasi baru yang datang di Topic yang yang dilanggani.

Fungsi Callback MQTT ketika ada pesan baru

```
def mqtt_subscription_callback(topic, message):
  # Get the current timestamp
  current_time = utime.localtime()

  timestamp = "{:04}-{:02}-{:02} {:02}:{:02}:{:02}".format(
    current_time[0], current_time[1], current_time[2],
    current_time[3], current_time[4], current_time[5]
  )
  # Debug print out of what was received over MQTT
  print (f'{timestamp} - Topic {topic} received message {message}')

# Before connecting, tell the MQTT client to use the callback
mqtt_client.set_callback(mqtt_subscription_callback)
mqtt_client.connect()
```

Setelah membuat fungsi callback, langkah selanjutnya adalah melakukan subscribe ke Topic yang Anda inginkan menggunakan fungsi:

```
mqtt_client.subscribe("<Topic_yang_dilanggani>")
```

Melakukan Subscribe pada topik yang dipilih

```
# Once connected, subscribe to the MQTT topic
mqtt_client.subscribe(mqtt_receive_topic)
print("Connected and subscribed")
```

Source Code untuk block try (seperti fungsi loop pada Arduino)

```
try:
  while True:
    # Infinitely wait for messages on the topic.
    # Note wait_msg() is a blocking call, if you're doing multiple things
    # on the Pico you may want to look at putting this on another thread.
    print(f'Waiting for messages on {mqtt_receive_topic}')
    mqtt_client.wait_msg()

# Jika ada masalah pada saat pengiriman
except Exception as e:
  print(f'MQTT error: {e}')

# Ketika program python dimatikan
finally:
  mqtt_client.disconnect()
```

Pada kode di atas, `mqtt_client.wait_msg()` akan menunggu pesan dari client MQTT yang memublikasikan pesan ke topik tersebut. Bila mendapatkan data, maka akan menghasilkan output sesuai dengan tindakan yang kita sudah definisikan pada fungsi `mqtt_subscription_callback()`.

LAMPIRAN MODUL 3

Source Code untuk Publish MQTT ke Mosquitto MQTT

Publish MQTT ke Mosquitto MQTT

```
import network
import time
from umqtt.simple import MQTTClient
import utime

# Isi nama Wi-Fi dan password yang terkoneksi bersama broker lokal di sini
wifi_ssid     = "<YOUR SSID WIFI>"
wifi_password = "<YOUR WIFI PASSWORD>"

# Isi sesuai dengan broker yang akan dipakai (Mosquitto, HiveMQ, AdafruitIO)
# dan lain-lain
mqtt_host     = "<MQTT HOST ADDRESS>"

# (Opsional) Bila menggunakan autentikasi
mqtt_username = "<USERNAME FROM MOSQUITTO MQTT>"
mqtt_password = "<PASSWORD FROM MOSQUITTO MQTT>"

# Isi sesuai dengan topik yang ingin di-publish
mqtt_publish_topic = "<YOUR mqtt_topic/feed_director>"

# Isi ID client random (harus unik tiap client)
mqtt_client_id = "uus9asndajsndiqiwd10wid01n1kwd"

# Angka yang akan dipublish
counter = 0

# Koneksi ke WiFi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(wifi_ssid, wifi_password)

while wlan.isconnected() == False:
  print('Waiting for connection...')
  time.sleep(1)

print("Connected to WiFi")

# Initialize our MQTTClient and connect to the MQTT server
mqtt_client = MQTTClient(
  client_id=mqtt_client_id,
  server=mqtt_host,
  # Bila menggunakan autentikasi
  user=mqtt_username,
  password=mqtt_password
)

mqtt_client.connect()

try:
  while True:
    # Generate some dummy data that changes every loop
    counter += 3

    # Get the current timestamp
    current_time = utime.localtime()
    timestamp = "{:04}-{:02}-{:02} {:02}:{:02}:{:02}".format(
      current_time[0], current_time[1], current_time[2],
      current_time[3], current_time[4], current_time[5]
    )

    # Publish the data to the topic!
```

```
    print(f'{timestamp} - Publish {counter}')
    mqtt_client.publish(mqtt_publish_topic, str(counter))

    # Delay a bit to avoid hitting the rate limit
    time.sleep(3)

# Jika ada masalah pada saat pengiriman
except Exception as e:
  print(f'MQTT error: {e}')

# Ketika program python dimatikan
finally:
  mqtt_client.disconnect()
```

Source Code untuk Subscribe MQTT dari Mosquitto MQTT

Subscribe MQTT dari Mosquitto MQTT

```
import network
import time
from umqtt.simple import MQTTClient
import utime

# Isi nama Wi-Fi dan password yang terkoneksi bersama broker lokal di sini
wifi_ssid     = "<YOUR SSID WIFI>"
wifi_password = "<YOUR WIFI PASSWORD>"

# Isi sesuai dengan broker yang akan dipakai (Mosquitto, HiveMQ, AdafruitIO)
# dan lain-lain
mqtt_host     = "<MQTT HOST ADDRESS>"

# (Opsional) Bila menggunakan autentikasi
mqtt_username = "<USERNAME FROM MOSQUITTO MQTT>"
mqtt_password = "<PASSWORD FROM MOSQUITTO MQTT>"

# Isi sesuai dengan topik yang ingin di-subscribe
mqtt_receive_topic = "<YOUR mqtt_topic/feed_director>"

# Isi ID client random (harus unik tiap client)
mqtt_client_id = "iij0dfq8f012nif2jnfijn"

# Koneksi ke WiFi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(wifi_ssid, wifi_password)

while wlan.isconnected() == False:
  print('Waiting for connection...')
  time.sleep(1)

print("Connected to WiFi")

# Initialize our MQTTClient and connect to the MQTT server
mqtt_client = MQTTClient(
  client_id=mqtt_client_id,
  server=mqtt_host,
  # Bila menggunakan autentikasi
  user=mqtt_username,
  password=mqtt_password
)

def mqtt_subscription_callback(topic, message):
  # Get the current timestamp
  current_time = utime.localtime()

  timestamp = "{:04}-{:02}-{:02} {:02}:{:02}:{:02}".format(
    current_time[0], current_time[1], current_time[2],
```

```
      current_time[3], current_time[4], current_time[5]
    )
    # Debug print out of what was received over MQTT
    print (f'{timestamp} - Topic {topic} received message {message}')

# Before connecting, tell the MQTT client to use the callback
mqtt_client.set_callback(mqtt_subscription_callback)
mqtt_client.connect()

mqtt_client.subscribe(mqtt_receive_topic)
print("Connected and subscribed")

try:
  while True:
    # Infinitely wait for messages on the topic.
    # Note wait_msg() is a blocking call, if you're doing multiple things
    # on the Pico you may want to look at putting this on another thread.
    print(f'Waiting for messages on {mqtt_receive_topic}')
    mqtt_client.wait_msg()

# Jika ada masalah pada saat pengiriman
except Exception as e:
  print(f'MQTT error: {e}')

# Ketika program python dimatikan
finally:
  mqtt_client.disconnect()
```